O P JINDAL UNIVERSITY

RAIGARH, CHHATTISGARH

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MINOR PROJECT REPORT

Project Title

Alpha X1: A Privacy-Preserving Browser Agent with On-Device Vision and Redaction

Submitted in partial fulfilment of the requirements for the Minor Project

Submitted by	[Student name(s)]
Roll / Enrollment Number	[Enter number(s)]
Programme / Semester / Section	[Enter details]
Academic Session	[20__–__]
Project Supervisor	[Name and designation]

Department of Computer Science and Engineering
O P Jindal University, Raigarh, Chhattisgarh

Certificate

This is to certify that the Minor Project Report entitled “Alpha X1: A Privacy-Preserving Browser Agent with On-Device Vision and Redaction” is a bona fide work carried out by [Student Name(s)], Roll/Enrollment No(s). [_____], of [Programme/Semester], Department of Computer Science and Engineering, O P Jindal University, Raigarh, Chhattisgarh, under my supervision during the academic session [20__–__].

Project Supervisor: _____ Date: _____

Head of Department: _____ Date: _____

Declaration

We declare that the work presented in this report entitled “Alpha X1: A Privacy-Preserving Browser Agent with On-Device Vision and Redaction” is our own work completed under the guidance of [Supervisor Name]. It has not been submitted, in whole or in part, for any other academic award. All sources used have been appropriately acknowledged.

Student Signature(s): _____ Date: _____

Acknowledgement

We express our sincere gratitude to our project supervisor, [Supervisor Name], for continuous guidance, constructive feedback and encouragement throughout this project. We are thankful to the Head of the Department and the faculty of the Department of Computer Science and Engineering, O P Jindal University, Raigarh, for providing the academic environment and resources needed to complete this work.

We also acknowledge the open-source communities behind FastAPI, React, Vite, ONNX Runtime, Tesseract, OpenCV’s YuNet and Ultralytics YOLO, whose tools made an on-device privacy pipeline practical, and our team members and friends for their support and testing help.

Abstract

Browser agents driven by large language models (LLMs) can read a web page and act on it — filling forms, clicking, navigating — but doing so normally means sending the raw page, and often a full screenshot, to a cloud provider. Such pages routinely contain passwords, payment details, national identifiers, contact information and faces, so the convenience of cloud reasoning comes at a direct privacy cost.

This project presents **Alpha X1 (Privacy Vision Agent)**, a Chrome extension (Manifest V3) paired with a FastAPI gateway that follows the principle *see locally, protect locally, reason in the cloud, validate locally, act locally*. A local pipeline combines DOM field rules, a checksum-aware regular-expression engine (Luhn, Verhoeff), Tesseract.js OCR, a YuNet face detector and a YOLO11n visual model running through ONNX Runtime Web. A fusion engine merges the findings and fails closed on high-risk types; a redactor replaces text with semantic tokens and blurs or blacks out image regions. A privacy gate verifies the sanitized context before anything is transmitted. The cloud model (Claude, OpenAI or a local Ollama model) receives only sanitized data and returns a structured action, which is validated twice — on the server and by a local security policy with a user-confirmation gate for dangerous actions — before the extension executes it.

The system is implemented in about 17,500 lines of TypeScript and Python and is verified by 358 automated tests (336 extension, 22 backend), all passing, with type-checking and continuous integration on every push. The report describes the design, implementation, testing and honest limitations of the system and outlines future work.

Keywords: privacy-preserving AI, browser agent, PII redaction, on-device vision, LLM security, Chrome extension

Table of Contents

Chapter 1: Introduction **6**
1.1 Background 6
1.2 Problem Statement 6
1.3 Aim and Objectives 6
1.4 Scope of the Project 7
1.5 Motivation and Significance 7
1.6 Organization of the Report 7
Chapter 2: Literature Review / Existing System **8**
2.1 Review of Related Work 8
2.2 Existing System and Limitations 8
2.3 Research / Development Gap 9
Chapter 3: Requirements and System Analysis **10**
3.1 Functional Requirements 10
3.2 Non-functional Requirements 10
3.3 Hardware and Software Requirements 11
3.4 Feasibility Analysis 11
3.5 Use Cases / User Requirements 11
Chapter 4: Proposed Methodology and System Design **12**
4.1 Proposed Approach 12
4.2 System Architecture 12
4.3 Module Description 13
4.4 Algorithm / Workflow 14
4.5 Database / Data Design 15
4.6 UI / Interface Design 15
Chapter 5: Implementation **16**
5.1 Development Environment 16
5.2 Implementation of Modules 16
5.3 Key Code / Configuration 17
5.4 Challenges and Solutions 18
Chapter 6: Testing and Results **19**
6.1 Testing Strategy 19
6.2 Test Cases and Results 19
6.3 Results / Screenshots 20
6.4 Evaluation and Discussion 22
Chapter 7: Conclusion and Future Scope **23**
7.1 Conclusion 23
7.2 Limitations 23
7.3 Future Enhancements 23
References **25**
Appendices **25**
Supervisor Evaluation / Approval **27**

List of Figures

Figure 4.1: System architecture showing local and cloud trust zones....12
Figure 4.2: End-to-end workflow of the Alpha X1 agent loop....14
Figure 6.1: Extension popup showing page analysis and backend/session status....21
Figure 6.2: Synthetic demo login page used for privacy evaluation....21

List of Tables

Table 2.1: Summary of related work and identified gaps....8
Table 3.1: Hardware and software requirements....11
Table 4.1: System modules, functions and outputs....13
Table 4.2: Sensitive-data categories recognised by the detection layer....13
Table 5.1: Development environment and technologies....16
Table 5.2: Action protocol and risk classification....17
Table 5.3: Representative challenges and how they were addressed....18
Table 6.1: Automated test summary by component....19
Table 6.2: Representative PII-detection test cases....20
Table 6.3: Representative action-security test cases....20
Table 6.4: Outcomes against objectives....22

List of Abbreviations

Abbreviation	Full form
API	Application Programming Interface
CI	Continuous Integration
CSS	Cascading Style Sheets
CVV	Card Verification Value
DOM	Document Object Model
IoU	Intersection over Union
LLM	Large Language Model
NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
ONNX	Open Neural Network Exchange
PAN	Permanent Account Number (India)
PII	Personally Identifiable Information
UPI	Unified Payments Interface
WASM	WebAssembly

Chapter 1: Introduction

1.1 Background

Large language models have made it possible to build *browser agents*: software that is given a goal in natural language ("log in and download my statement"), observes a web page, decides on the next step and carries it out by clicking, typing, scrolling or navigating. Commercial offerings and research prototypes of this kind generally observe the page through the DOM, through screenshots, or both, and send that observation to a cloud-hosted model [1], [2].

The pages an agent is useful on are precisely the pages that hold sensitive information: banking and payment forms, e-mail inboxes, government and identity portals, HR and health systems. Passwords, card numbers, Aadhaar and PAN-style identifiers, UPI addresses, phone numbers, postal addresses and photographs of people all appear on screen. Regulations such as India's Digital Personal Data Protection Act, 2023 [3] and the wider security community's guidance on LLM applications [4] make clear that this data must be handled with care.

Alpha X1 explores a different split of responsibilities. Perception and protection happen on the user's device; only a sanitized description of the page ever reaches the cloud model; and every action the model proposes is checked locally before it is executed.

1.2 Problem Statement

Existing LLM browser agents forward raw page content and unredacted screenshots to a third-party service. A user who wants automation therefore has to trust that service with every credential, identifier and face visible in the tab. At the same time, an agent that executes whatever the model returns is exposed to mistakes and to prompt-injection from page content [4]. There is a need for an agent architecture that (a) keeps raw sensitive data on the device, (b) still gives the model enough sanitized context to act usefully, and (c) treats the model's output as untrusted until it has been validated.

1.3 Aim and Objectives

Aim: To design and implement a browser agent that performs LLM-driven web automation while detecting and removing sensitive information locally, so that the cloud model sees only sanitized context and all proposed actions are validated before execution.

Objectives:

- Build a Chrome Manifest V3 extension that captures the DOM and a screenshot of the active tab and executes browser actions.
- Implement a fully local detection layer combining DOM rules, regular-expression/checksum matching, OCR, face detection and a general visual model.
- Fuse detector outputs and redact text, form fields and image regions in a fail-closed manner, with a privacy gate that blocks transmission when verification fails.
- Develop a FastAPI gateway that manages WebSocket sessions and routes sanitized context to interchangeable reasoning providers (Claude, OpenAI, Ollama) using a strict action protocol.
- Enforce layered action security: server-side validation, a local risk-based policy, user confirmation for dangerous actions, a kill switch and session time-outs.
- Verify the system with an automated test suite and continuous integration, and provide an evaluation harness for detection accuracy.

1.4 Scope of the Project

The project covers the end-to-end prototype: the extension (capture, local privacy pipeline, side-panel and popup UI, action executor, security layer), the backend gateway and provider adapters, a synthetic demo site and an evaluation framework. The intended users are individuals who want to automate web tasks without exposing private data to a cloud model, and developers researching privacy-preserving agents.

Assumptions and boundaries. The target browser is Chromium-based (Manifest V3). Detection is designed for English-language pages and for common Indian and international identifier formats. The project does not claim to verify that a detected identifier is genuine — it recognises its *shape* — and it does not guarantee perfect detection; it is a risk-reduction layer. Operation at production scale (multi-user hosting, persistent storage, authentication of users) is outside the scope of this minor project.

1.5 Motivation and Significance

Privacy is the main obstacle to using AI agents on real accounts. By proving that useful automation can be achieved with on-device sanitization, the project demonstrates a practical pattern that individuals, small organizations and regulated sectors can adopt. It also exercises a broad range of computer-science skills — browser extension engineering, computer vision inference in WebAssembly, real-time protocols, secure design and software testing — in a single integrated system.

1.6 Organization of the Report

- **Chapter 2** reviews related work and identifies the gap the project addresses.
- **Chapter 3** states the functional and non-functional requirements, platform needs, feasibility and use cases.
- **Chapter 4** presents the methodology, architecture, modules, workflow, data design and interface design.
- **Chapter 5** describes the implementation, key code and the challenges met.
- **Chapter 6** reports the testing strategy, test cases, results and discussion.
- **Chapter 7** concludes, lists limitations and suggests future enhancements.

Chapter 2: Literature Review / Existing System

2.1 Review of Related Work

The related work falls into three groups: LLM agents that operate user interfaces, PII-detection and anonymization tooling, and the model and OCR components that make local detection feasible. Table 2.1 summarizes the most relevant items.

Table 2.1: Summary of related work and identified gaps

Author / Year	Work / Approach	Key Findings	Limitation / Research Gap
Anthropic, 2024 [1]	Computer-use capability: model views screenshots and issues mouse/keyboard actions	LLMs can operate general GUIs from pixels alone	Screenshots and page content are sent to the cloud unredacted; no local privacy stage
OpenAI, 2025 [2]	Operator: browsing agent that observes pages and acts on them	Shows commercial viability of web agents; confirmation for sensitive steps	Cloud-side perception of raw pages; privacy relies on provider policy
Microsoft Presidio [5]	Open-source PII detection and anonymization for text and images	Mature recognizers and anonymizers; extensible	A general library, not integrated into a browser agent loop or action validation
Smith, 2007 [6]	Tesseract OCR engine	Open-source, accurate text recognition; now available in WebAssembly	Needs careful local packaging so no assets are fetched from a CDN
Wu et al., 2023 [7]	YuNet lightweight face detector	Millisecond-level face detection suitable for CPU/edge use	Detects faces only; no notion of documents or screens
Ultralytics / Lin et al., 2024 / 2014 [8], [9]	YOLO11 object detector trained on COCO	Fast general object detection, exportable to ONNX	Generic classes (person, laptop, phone, tv, book) must be mapped to privacy risks
OWASP, 2023 [4]	Top 10 risks for LLM applications	Identifies prompt injection and excessive agency as key risks	Gives guidance only; agents need concrete action validation and confirmation

2.2 Existing System and Limitations

Current cloud browser agents follow an “observe raw, reason in cloud” design. Their limitations, as they relate to this project, are:

- **Raw data exposure:** passwords typed or displayed, card numbers, identity numbers and faces in screenshots are transmitted to the provider.
- **Single point of trust:** protection depends entirely on a vendor’s retention and access policies, which the user cannot verify.
- **Unchecked actions:** the model’s output is executed directly; malicious page text can steer the agent (prompt injection), and navigation to dangerous schemes or local addresses is not systematically blocked.
- **Limited auditability:** the user rarely sees exactly what was sent, nor can they stop a runaway loop with a reliable kill switch.

General PII tools such as Presidio address detection but are not designed for in-browser, real-time use, do not combine DOM semantics with visual detectors, and do not govern what the agent may do next.

2.3 Research / Development Gap

What is missing is an integrated, on-device system that (i) detects sensitive content across DOM, text, images and faces, (ii) redacts it before any network transmission and proves the context is clean via a gate, (iii) lets the cloud model act on the sanitized view, and (iv) validates and confirms the resulting actions. Alpha X1 addresses this gap as a working, tested prototype that also exposes an “outbound data viewer” so the user can inspect exactly what leaves the device.

Chapter 3: Requirements and System Analysis

3.1 Functional Requirements

1. **FR1 – Page capture:** capture the active tab's DOM structure (interactive elements with stable local identifiers) and a screenshot.
2. **FR2 – Sensitive-data detection:** detect sensitive items in form fields (by type, `autocomplete`, name, label), in text (e-mail, phone, card, Aadhaar-like, PAN-like, UPI, IP address), in images through OCR, and faces and document-like objects in screenshots.
3. **FR3 – Fusion and redaction:** merge duplicate findings, apply a fail-closed risk policy, and redact text with semantic tokens, mask field values, and blur or black out image regions.
4. **FR4 – Privacy gate:** block transmission when verification fails or high-risk findings remain unresolved; let the user review findings and the outbound payload.
5. **FR5 – Cloud reasoning:** send only the sanitized context and task over a WebSocket session to the backend, which obtains the next action from a selectable provider (Claude, OpenAI or Ollama).
6. **FR6 – Action protocol:** support the actions `click`, `type`, `select`, `scroll`, `wait`, `navigate` and `finish` in a strict structured format.
7. **FR7 – Action validation:** validate every action on the server and again in the extension against a risk policy; require explicit user confirmation for dangerous actions.
8. **FR8 – Agent loop:** execute actions, report results back, maintain a bounded action history and warn after repeated consecutive failures.
9. **FR9 – Safety controls:** provide a kill switch, idle-session time-out and reconnection with heartbeat.
10. **FR10 – Evaluation:** measure detection precision/recall, redaction, resource use and latency using synthetic data.

3.2 Non-functional Requirements

- **Security and privacy:** raw sensitive values never leave local memory; logs are scrubbed of secrets by a privacy-safe logger; no local-processing component may fetch assets from a CDN.
- **Reliability:** the system fails closed — if detection is uncertain on a high-risk type, it redacts; if the gate cannot verify, it blocks; if no confirmation arrives within the time-out, the action is denied.
- **Performance:** detection stages are timed individually and run asynchronously with time-outs so a stalled model cannot hang the UI.
- **Usability:** a popup for quick status and a side panel that shows detections, the sanitized context, the cloud decision and the execution status.
- **Maintainability and portability:** typed TypeScript and Python, modular detectors behind interfaces, provider abstraction, and CI on every push.
- **Extensibility:** new PII rules are registered in a rule registry without changing the engine.

3.3 Hardware and Software Requirements

Table 3.1: Hardware and software requirements

Item	Minimum	Recommended
Processor	Dual-core 64-bit CPU	Quad-core or better (WebAssembly inference)
Memory	4 GB RAM	8 GB RAM or more
Storage	1 GB free (including model files)	2 GB free
Browser	Chromium-based browser with Manifest V3 and Side Panel support	Latest stable Google Chrome
Backend runtime	Python 3.12	Python 3.12 with virtual environment
Extension build	Node.js 20 and npm	Node.js 20 LTS
Network	Needed only for the cloud provider API	Not needed with a local Ollama model
Accounts / keys	API key for Claude or OpenAI (or a local Ollama install)	—

3.4 Feasibility Analysis

Technical feasibility. All components rely on mature open-source technology: Chrome extension APIs, React, ONNX Runtime Web, Tesseract.js, FastAPI and standard LLM APIs. The prototype demonstrates that inference for face and object detection and OCR runs on the user's device through WebAssembly.

Operational feasibility. The user installs an unpacked extension and runs a local backend with a single command; the interface shows what was detected and what will be sent, which keeps the user in control.

Schedule and economic feasibility. The work was developed incrementally over about three weeks of repository activity (9–30 September 2026) using free tools. The only running cost is the optional cloud API usage, which can be eliminated by using a local Ollama model.

3.5 Use Cases / User Requirements

The primary actor is the **end user** who delegates a browsing task; secondary actors are the **cloud reasoning provider** and the **web page** being automated.

- **UC1 – Run a task safely:** the user enters a task; the extension captures and sanitizes the page, the user reviews the outbound data, and the agent proceeds step by step.
- **UC2 – Review detections:** the user inspects overlays, the list of findings and the sanitized context before anything is sent.
- **UC3 – Approve a dangerous action:** for `navigate` or `finish` the user is asked to approve or reject within 60 seconds.
- **UC4 – Stop immediately:** the user triggers the kill switch to terminate the loop.
- **UC5 – Evaluate:** a developer runs the benchmark to obtain detection metrics on synthetic data.

Chapter 4: Proposed Methodology and System Design

4.1 Proposed Approach

The approach is a **five-stage trust pipeline**: SEE LOCALLY → PROTECT LOCALLY → REASON IN CLOUD → VALIDATE LOCALLY → ACT LOCALLY. The central design decision is that the cloud model is a *reasoner over sanitized data*, never a perceiver of raw data. Three principles guide the design:

- **Redact the value, keep the structure.** The model must still understand the page, so output reads "Email: [EMAIL]" rather than "[REDACTED PAGE]".
- **Fail closed.** When the system is uncertain about a high-risk type (password, card, CVV, Aadhaar-like, PAN-like, account number, face) it redacts rather than lets the value through.
- **Never trust the model.** Every returned action is untrusted input that must pass validation and, if dangerous, human confirmation.

4.2 System Architecture

The system has two deployable parts: the browser extension (the local trust zone) and the FastAPI gateway (the boundary to the cloud zone). Figure 4.1 shows the components and the one-way flow of sanitized data.

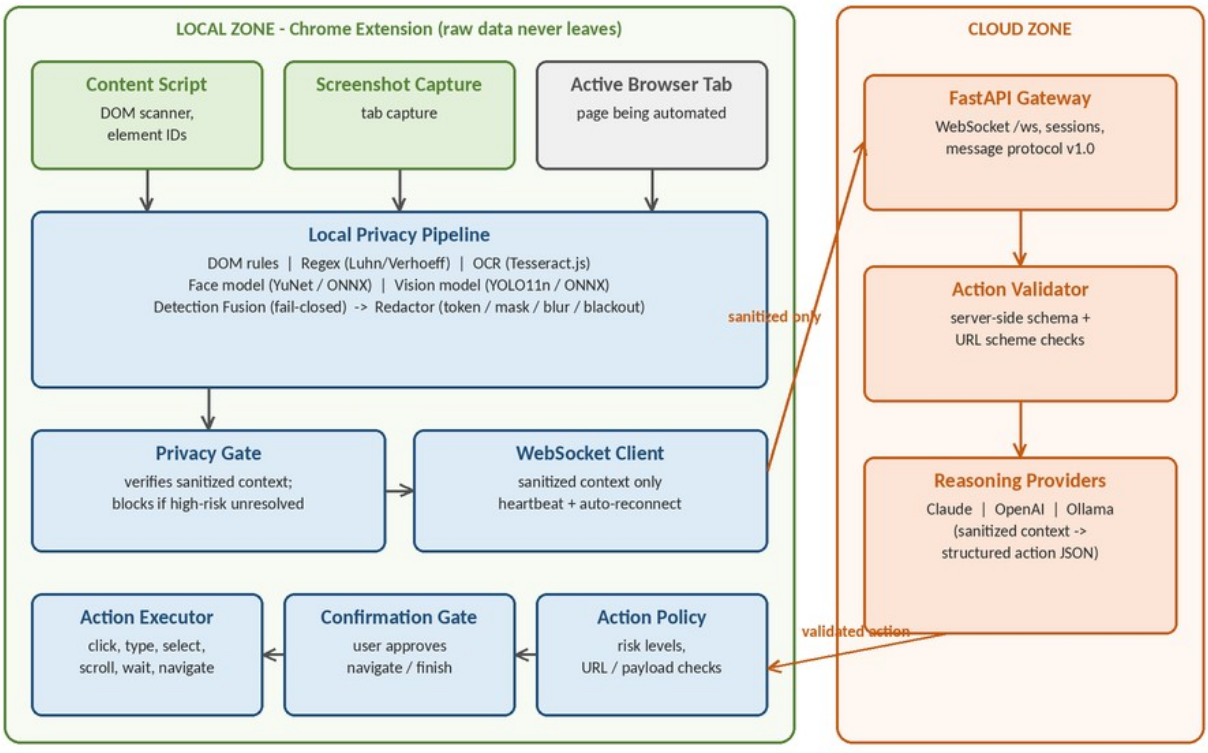


Figure 4.1: System architecture showing local and cloud trust zones

The content script and screenshot helper feed the local privacy pipeline; its output is checked by the privacy gate and sent through a WebSocket client to the gateway. The gateway validates and forwards the sanitized context to the selected reasoning provider and returns a validated structured

action, which passes through the local action policy and confirmation gate before the executor performs it in the tab.

4.3 Module Description

Table 4.1: System modules, functions and outputs

Module	Description / Function	Expected Output
DOM scanner / content script	Walks the page, records interactive elements with local IDs, labels and field metadata	Element list for the agent and for DOM-rule detection
DOM rules detector	Classifies fields from `autocomplete` tokens, input type, name/id/label keywords	Findings with confidence 1.0 for semantic fields
Regex detector	Rule registry for e-mail, phone, card (Luhn), Aadhaar-like (Verhoeff), PAN-like, UPI, IPv4/IPv6	Text-span findings
OCR (Tesseract.js)	Reads text baked into images/canvas; digit-recovery pass for numeric identifiers; assets loaded locally	Text with boxes, fed to the regex engine
Face detector (YuNet)	ONNX face model with greedy NMS; detects faces without identifying them	Face bounding boxes
Vision model (YOLO11n)	COCO objects mapped to privacy classes (person, book, laptop, cell phone, tv)	Document/face-like regions
Detection fusion	Groups findings by element, text-span overlap or IoU > 0.5; ranks sources; fail-closed policy	One normalized finding list
Redactor	Token replacement for text, masking of fields, blur or blackout of image regions	Sanitized text and screenshot
Privacy gate / outbound validator	Decides safe/blocked/idle; re-validates the outgoing payload independently of the UI	Gate state and send permission
Backend gateway	FastAPI app, WebSocket endpoint, sessions, heartbeat, protocol v1.0	Validated actions for the extension
Providers	Claude, OpenAI and Ollama adapters behind one `BaseProvider` interface	Structured `ActionResponse`
Action policy & confirmation gate	Risk levels, payload and URL checks, user approval for dangerous actions	Allow / deny decision
Action executor	Performs click, type, select, scroll, wait, navigate and finish in the tab	Action result reported to backend
Security utilities	Kill switch, session manager, privacy-safe logger	Safe termination and scrubbed logs
Evaluation framework	Synthetic datasets, privacy and visual evaluators, metrics collector, benchmark runner	Precision/recall/F1 and resource metrics

Table 4.2: Sensitive-data categories recognised by the detection layer

Category	Detected by	Default redaction
Password, card CVV	DOM rules (field semantics)	Mask field value

Category	Detected by	Default redaction
Credit card number, card expiry	DOM rules, regex (Luhn), OCR	Token `[CREDIT_CARD]` / `[CARD_EXPIRY]`
E-mail, phone, person name, address	DOM rules, regex, OCR	Tokens `[EMAIL]`, `[PHONE]`, `[PERSON]`, `[ADDRESS]`
Aadhaar-like, PAN-like, UPI ID, account number	DOM rules, regex (Verhoeff for Aadhaar shape), OCR	Tokens such as `[AADHAAR_LIKE_ID]`
IP address (v4/v6), other sensitive IDs	Regex, OCR	Tokens `[IP_ADDRESS]`, `[REDACTED]`
Faces	YuNet face model, YOLO `person` class	Blur
Documents / screens	YOLO classes (book, laptop, cell phone, tv)	Blackout

Type names such as `AADHAAR_LIKE_ID` and `PAN_LIKE_ID` are deliberate: pattern matching identifies the shape of an identifier, never its real-world validity.

4.4 Algorithm / Workflow

Figure 4.2 shows the agent loop. Each iteration runs the six steps below until the model returns `finish`, the kill switch is activated or the session times out.

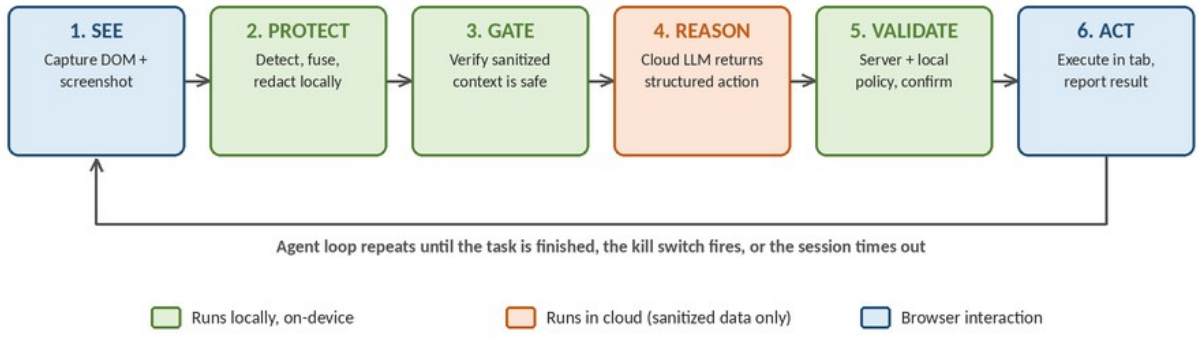


Figure 4.2: End-to-end workflow of the Alpha X1 agent loop

Pseudocode of one iteration:

```
loop:
  if killSwitch.active or session.expired: stop
  dom, shot  <- capture(activeTab)
  raw        <- DOMrules(dom) + Regex(texts) + OCR(images)
                + Face(shot) + Vision(shot)
  findings   <- Fusion(raw)              # merge, rank, fail-closed
  clean      <- Redact(dom, shot, findings)
  if not PrivacyGate(clean, findings): block; ask user; continue
  action     <- Backend.reason(clean, task, history)   # cloud
  ok, why    <- ServerValidator(action)
```

```
verdict    <- ActionPolicy(action)     # SAFE/MODERATE/DANGEROUS/BLOCKED
if verdict.requiresConfirmation and not user.confirm(60s): deny
result     <- Executor.run(action)
Backend.report(result)                 # bounded history, failure count
if action == finish: stop
```

Fusion rules. Two findings merge when they share an element ID, have overlapping text spans or overlapping boxes (IoU > 0.5). The merged finding keeps the type of the higher-ranked source (DOM > regex > OCR/face > vision), the maximum confidence, and records any disagreement. Low-confidence findings are dropped unless they are high-risk types, which are promoted to "redact anyway".

4.5 Database / Data Design

The prototype deliberately uses no persistent database: raw sensitive values are kept only in local memory and are never serialized out. The key data structures are:

- **`PrivacyFinding`** — `type`, `source`, `confidence`, optional `bbox`, `elementId`, `textSpan`, local-only `rawValue`, `replacement`, `strategy` and `detail`. A `toWireFinding` function strips `rawValue` before anything leaves the extension.
- **`MessageEnvelope`** (protocol v1.0) — `session_id`, `message_id`, `type` (context, action, action_result, error, heartbeat, page_changed), `timestamp` and a `payload` dictionary, validated with Pydantic.
- **`SessionData`** (backend, in memory) — creation time, last heartbeat, message counters, a history of at most 20 action results, a consecutive-failure counter and pending actions. Sessions idle for more than five minutes are considered stale.
- **`ActionResponse`** — `action_type`, `target_id`, `value`, `option`, `direction`, `amount`, `duration_ms`, `url`, `confidence`, `reason`.

4.6 UI / Interface Design

The extension offers two surfaces. The **popup** (Figure 6.1) gives quick access to page information, DOM analysis, screenshot capture and backend/session status. The **side panel** is the main workspace and is built from React components such as the processing pipeline view, detection overlay and list, sanitized-context viewer, split original/redacted screenshot view, privacy gate, outbound-data viewer, cloud decision, action validation, pending-action confirmation, execution status, session timeline and metrics panel. The design goal is transparency: the user can see what was found, what will be sent and what the agent intends to do.

Chapter 5: Implementation

5.1 Development Environment

Table 5.1: Development environment and technologies

Category	Technology	Purpose
Languages	TypeScript 5.3 (extension), Python 3.12 (backend)	Type-safe extension code; async backend
Extension framework	Chrome Manifest V3, React 18, Vite 5, lucide-react	Service worker, content script, popup and side panel
Local ML / vision	onnxruntime-web 1.19, Tesseract.js 7, YuNet (ONNX), YOLO11n (ONNX)	On-device OCR, face and object detection
Backend	FastAPI 0.104, Uvicorn, websockets 12, Pydantic 2	Gateway, WebSocket sessions, schema validation
Cloud providers	anthropic SDK, openai SDK, httpx (Ollama)	Reasoning adapters
Testing	Vitest 1 + jsdom (extension), pytest 7 (backend)	Automated unit/integration tests
Quality tools	ESLint, `tsc --noEmit`, GitHub Actions CI	Lint, type-check, test and build on each push/PR
Version control	Git and GitHub (repository `princeraj2572/alpha-x1`)	83 commits across 9–30 Sep 2026

The repository is organized as `privacy-vision-agent/` with `backend/`, `extension/` and `demo-site/` directories; the extension source is split into `privacy/`, `security/`, `executor/`, `communication/`, `background/`, `evaluation/`, `scanner/`, `capture/` and `ui/`. It contains roughly 17,500 lines of TypeScript, TSX and Python.

5.2 Implementation of Modules

Local privacy pipeline

`PrivacyPipeline` orchestrates the detectors, each of which is optional and individually timed: DOM rules, regex over supplied text blocks, OCR, face detection and vision detection, followed by fusion and redaction. Each detector sits behind an interface (for example `OcrEngine`, `FaceModel`) so that a stub can replace a model in tests and a model that fails to load degrades gracefully instead of crashing the pipeline. The pipeline returns the fused findings, visual regions, the redacted screenshot and text, a safe-to-log report and per-stage timings.

Detection engines

The DOM-rule detector classifies each field using, in order, the standardized `autocomplete` token, the input type and finally keyword heuristics over name, id, placeholder and label. The regex engine is a registry of `PiiRule` objects with optional validators; card candidates must pass the Luhn checksum and Aadhaar-like candidates the Verhoeff checksum, which sharply reduces false positives such as order numbers and version strings. OCR uses Tesseract.js with every asset (worker, WebAssembly core, language data) loaded from extension-local URLs, because the

library's default of fetching from a public CDN would contradict the project's premise. Face and vision models run through ONNX Runtime Web with execution-provider fallback.

Fusion, redaction and the privacy gate

The fusion engine implements the rules of Section 4.4. The redactor applies text spans right-to-left so offsets remain valid, masks DOM values, and blurs or blacks out pixel regions. The privacy gate is a set of pure functions: the UI renders its result, but the transport layer independently re-validates the sanitized context before sending, so safety never depends on the UI.

Backend gateway and providers

The FastAPI application exposes health and status endpoints and a WebSocket endpoint that accepts envelopes, tracks sessions, sends heartbeats and relays validated actions. `ReasoningService` lazily creates the provider selected by the `AI_PROVIDER` environment variable (Claude by default, OpenAI or Ollama). All providers implement `BaseProvider.reason()` and return a common `ActionResponse`. Session state records recent action results and warns the extension after three consecutive failures.

Action security

Actions are classified by `ActionPolicyValidator` as in Table 5.2. The server-side `ActionValidator` enforces required fields and allowed URL schemes independently, so a compromised or confused extension, or a bad model response, is caught at either layer.

Table 5.2: Action protocol and risk classification

Action	Risk level	Key checks
click, scroll, wait	SAFE	`target_id` or direction required; scroll `amount` must be numeric and ≤ 1000; wait 100–10,000 ms
type, select	MODERATE	Value must be a string; suspicious script content rejected
navigate	DANGEROUS (confirmation required)	URL required; blocks `javascript:`, `data:`, `file://`, `about:`, `blob:`; blocks localhost/127.0.0.1/0.0.0.0; malformed URLs rejected
finish	DANGEROUS (confirmation required)	User must approve completion
Any action	BLOCKED if invalid	Unknown type or payload larger than 10 KB is blocked

Dangerous actions are held by the `ConfirmationGate`, which returns a promise that resolves only when the user answers (or denies automatically after 60 seconds). The `KillSwitch` can stop the loop at any time and notifies subscribers; the `SessionManager` enforces a ten-minute idle time-out and bounded reconnection attempts; `SafeLogger` redacts secrets such as passwords, tokens, cookies and API keys from log data.

5.3 Key Code / Configuration

The following excerpts show two central mechanisms. First, the Luhn checksum used to reduce false positives on card-like numbers:

```
export function luhnValid(value: string): boolean {
```

```
  const digits = value.replace(/\D/g, '');
  if (digits.length < 12 || digits.length > 19) return false;
  let sum = 0, double = false;
  for (let i = digits.length - 1; i >= 0; i--) {
    let d = digits.charCodeAt(i) - 48;
    if (double) { d *= 2; if (d > 9) d -= 9; }
    sum += d; double = !double;
  }
  return sum % 10 === 0;
}
```

Second, the fail-closed set of high-risk types that fusion and the privacy gate treat conservatively:

```
export const HIGH_RISK_TYPES: ReadonlySet<PrivacyType> = new Set([
  PrivacyType.PASSWORD, PrivacyType.CREDIT_CARD, PrivacyType.CARD_CVV,
  PrivacyType.AADHAAR_LIKE_ID, PrivacyType.PAN_LIKE_ID,
  PrivacyType.ACCOUNT_NUMBER, PrivacyType.FACE,
]);
```

Configuration is supplied through a `.env` file (copied from `.env.example`): `AI_PROVIDER` (anthropic, openai or ollama), the corresponding API key and model, the backend URLs and the log level. Real credentials are never committed. Complete source is available in the project repository.

5.4 Challenges and Solutions

Table 5.3: Representative challenges and how they were addressed

Challenge	Solution
Cloud model must act on elements it cannot see raw; `click`/`type`/`select` actions were not resolving to real elements	Introduced stable local `target_id` values and fixed the resolution so each action targets the intended element
Confirmation was computed and displayed but did not actually pause execution	Added a dedicated `ConfirmationGate` that blocks until the user decides or a time-out occurs, with unit tests
An unauthenticated reasoning endpoint and a popup “Ask Agent” button could bypass the privacy pipeline	Removed both so the only route to the cloud is the sanitized, gated WebSocket flow
Tesseract and ONNX defaults fetch assets from CDNs, violating the local-only principle	Packaged assets inside the extension and passed explicit local URLs
Models unavailable in tests (browser-only runtime) risked fake “pass” results	Evaluators report `measured: false` when a model cannot run instead of fabricating a score
Idle-session time-out existed but was not enforced; duplicate handlers and reconnect-after-disconnect bugs in the WebSocket client	Wired the session manager into the agent loop and fixed handler registration and `disconnect()` behaviour
Dead or orphaned code (old vision engine, unused privacy service, placeholder dashboard) increased the attack surface	Removed unused modules and consolidated evaluation onto one maintained dataset

Chapter 6: Testing and Results

6.1 Testing Strategy

Testing combines automated unit and integration tests, static checks and continuous integration:

- **Unit tests** for every detector, fusion, redaction, security module, state store and loader (Vitest with jsdom for the extension; pytest for the backend).
- **Integration tests** for the pipeline orchestrator and for the WebSocket action flow (session, validation, history and failure handling).
- **Dataset-driven tests** using a synthetic, privacy-safe dataset of 21 text cases and 19 form-field cases, including negatives that must *not* trigger (order numbers, version strings, promo codes).
- **Static checks:** `tsc --noEmit` and ESLint for the extension.
- **Continuous integration:** a GitHub Actions workflow runs type-check, lint, tests and the production build for the extension, and pytest for the backend, on every push and pull request.

Only synthetic and industry-standard test values are used (for example the card test numbers 4111 1111 1111 1111 and 5555 5555 5555 4444, and reserved `example.invalid` domains); no real personal data appears in the repository.

6.2 Test Cases and Results

The full automated suite was executed for this report on the project's main code base. All 358 tests passed and the TypeScript type-check completed without errors. Table 6.1 summarizes the results by component.

Table 6.1: Automated test summary by component

Component	Test files	Tests	Result
Privacy detection (regex, DOM rules, detector facade, fusion, redactor, OCR, face, vision, pipeline, async utils)	12	131	All passed
Security (action policy, kill switch, safe logger, session manager)	4	72	All passed
Executor, confirmation gate and WebSocket client	3	25	All passed
UI state (store, messaging, privacy gate, outbound validator, loaders, pipeline runner)	8	63	All passed
DOM scanner	1	23	All passed
Evaluation framework	3	22	All passed
Extension total	31	336	All passed
Backend (providers, sessions, WebSocket action flow)	5	22	All passed
Overall	36	358	All passed

Representative functional test cases are listed in Tables 6.2 and 6.3. Expected and actual outputs agree in every case listed.

Table 6.2: Representative PII-detection test cases

Input	Expected output	Actual	Status
“Card on file 4111 1111 1111 1111”	CREDIT_CARD detected	CREDIT_CARD	Pass
“Aadhaar 2338 5972 6560” (Verhoeff-valid, synthetic)	AADHAAR_LIKE_ID detected	AADHAAR_LIKE_ID	Pass
“PAN: ABCDE1234F”	PAN_LIKE_ID detected	PAN_LIKE_ID	Pass
“Send to testuser@oksbi”	UPI_ID detected	UPI_ID	Pass
Combined text with name, e-mail, phone and card	EMAIL, PHONE, CREDIT_CARD	All three found	Pass
“Version 4.11.2024 of the doc”	No finding	No finding	Pass
“Order #100200300 shipped”	No finding	No finding	Pass
Field `type=password`, `autocomplete=new-password`	PASSWORD	PASSWORD	Pass
Field `autocomplete=cc-csc`	CARD_CVV	CARD_CVV	Pass
Field name `search`, placeholder “Search products”	Not sensitive	Not sensitive	Pass

Table 6.3: Representative action-security test cases

Input	Expected output	Actual	Status
`navigate` with an https URL	Valid, DANGEROUS, confirmation required	As expected	Pass
`navigate` with `javascript:` URL (any letter case)	Rejected as BLOCKED	Rejected	Pass
`navigate` with malformed URL	Rejected without throwing	Rejected	Pass
Unknown action type	BLOCKED (not defaulted to SAFE)	BLOCKED	Pass
Payload larger than 10 KB	Rejected	Rejected	Pass
`scroll` with amount over 1000	Rejected	Rejected	Pass
`finish` action	DANGEROUS, confirmation required	As expected	Pass
Confirmation not answered within time-out	Denied automatically	Denied	Pass
Backend: consecutive action failures reach threshold	Warning raised to extension	Warning raised	Pass

6.3 Results / Screenshots

Figure 6.1 shows the extension popup running against a page, with page information, DOM analysis, screenshot capture and backend/session status. Figure 6.2 shows the bundled demo login page, a synthetic page with credential and personal-information fields used for development and privacy evaluation.

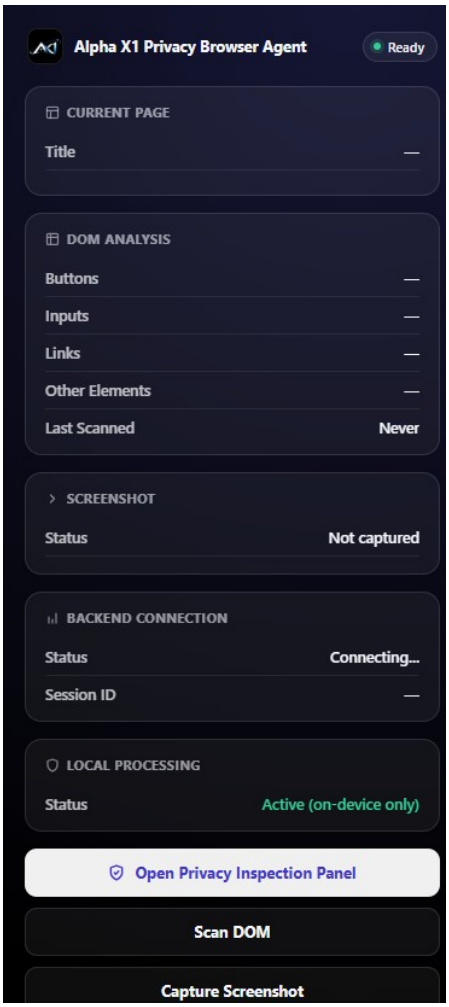


Figure 6.1: Extension popup showing page analysis and backend/session status

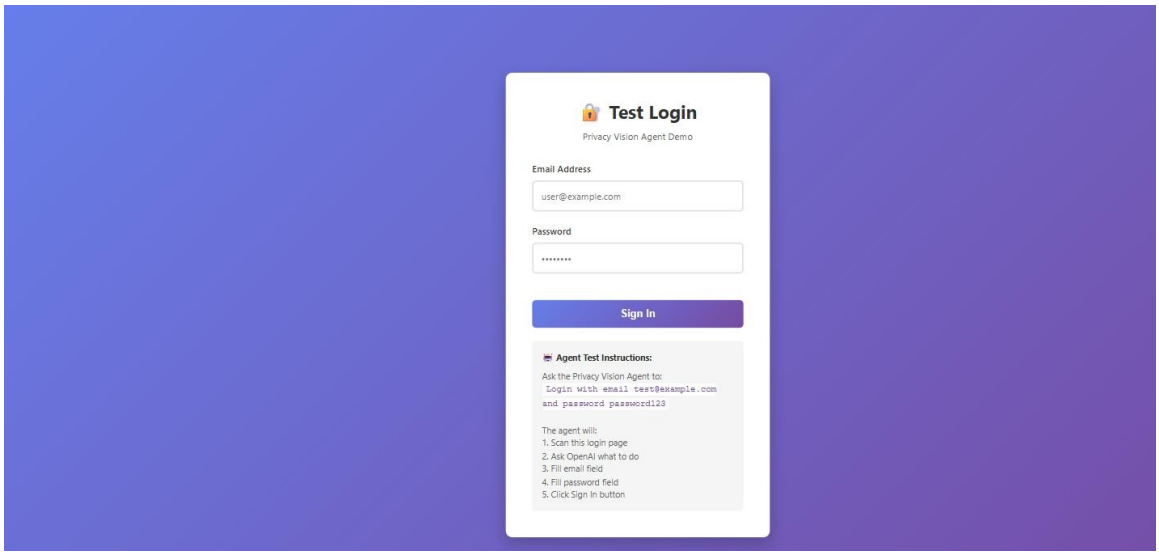


Figure 6.2: Synthetic demo login page used for privacy evaluation

6.4 Evaluation and Discussion

Table 6.4: Outcomes against objectives

Objective	Outcome
Capture DOM and screenshot; execute actions	Achieved: content script, screenshot helper and executor implemented and unit-tested
Local multi-detector privacy layer	Achieved: DOM, regex, OCR, face and vision detectors with graceful fallback
Fusion, fail-closed redaction, privacy gate	Achieved: implemented and tested; gate re-validated at the transport layer
Gateway with interchangeable providers	Achieved: Claude, OpenAI and Ollama adapters behind one interface
Layered action security	Achieved: server validation, local policy, confirmation gate, kill switch, session time-out
Testing and CI	Achieved: 358 tests passing; CI runs type-check, lint, tests and build

Discussion and honest limitations of the evaluation. The automated tests show that each component behaves as specified on synthetic data. They do **not** establish detection accuracy on real-world pages. In particular, the ONNX face and object models and Tesseract's WebAssembly runtime are browser-only and are not exercised under Node-based tests, so visual-detection accuracy is reported as *not measured* by the evaluators in that environment rather than given a made-up score. A real precision/recall figure for the visual detectors requires the labelled fixture images, which are intentionally not bundled for licensing and privacy reasons and are fetched on demand with `npm run fetch:visual-fixtures`. We therefore report test status, not accuracy percentages, and recommend a measured browser-based evaluation as future work.

Chapter 7: Conclusion and Future Scope

7.1 Conclusion

This project set out to show that an LLM browser agent can be useful without being given raw sensitive data. Alpha X1 realises the "see locally, protect locally, reason in cloud, validate locally, act locally" design as a working prototype: a Manifest V3 extension with a multi-detector, fail-closed privacy pipeline and an inspectable privacy gate, a FastAPI gateway with interchangeable cloud providers, and a layered action-security model with user confirmation, a kill switch and session controls. The system is covered by 358 passing automated tests and continuous integration, and its development history documents several security-relevant fixes (such as removing paths that bypassed the privacy pipeline).

The main contribution is the integration: the individual techniques (regex with checksums, OCR, face and object detection, schema validation) are known, but combining them so that sanitization is enforced before transmission and action validation after reasoning, with the user able to see both, is what makes the agent safer to use on real accounts.

7.2 Limitations

- Detection is probabilistic and rule-based; novel identifier formats, handwritten or heavily distorted text, and unusual layouts can be missed. Detection is tuned for English and common Indian and international formats.
- Identifier checks establish shape and checksum validity only, not that a number is real or belongs to anyone.
- The face and vision models and OCR assets are not bundled in the repository and must be provided locally; accuracy on real pages has not yet been measured systematically.
- The vision model uses generic COCO classes mapped to privacy classes, which is coarse (for example any "laptop" is treated as a potential document).
- Backend sessions are held in memory in a single process; there is no user authentication, persistence or horizontal scaling.
- Redacted context can still reveal structure and the user's task to the cloud provider; the system protects values, not the fact of using a service.
- The extension requests broad host permissions (`<all_urls>`) because it must operate on arbitrary pages.

7.3 Future Enhancements

- Run a browser-based benchmark on labelled real-world screenshots and publish precision/recall per category, including latency and memory on typical laptops.
- Train or fine-tune a purpose-built detector for document types (ID cards, bank statements) instead of mapping generic COCO classes.
- Add stronger prompt-injection defences, such as separating page text from instructions and allow-listing domains per task.
- Support more languages and identifier formats, and user-defined redaction rules.

- Add authentication, persistent encrypted audit logs and multi-user deployment to the backend; support fully offline reasoning with a local model by default, and ship a packaged extension build with a model-download onboarding flow.

References

Note: all sources below were consulted for the concepts used in this report. Please verify page numbers and URLs against the originals before submission.

[1] Anthropic, “Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku,” Anthropic News, Oct. 2024. [Online]. Available: https://www.anthropic.com/news/3-5-models-and-computer-use

[2] OpenAI, “Introducing Operator,” OpenAI, Jan. 2025. [Online]. Available: https://openai.com/index/introducing-operator/

[3] Government of India, “The Digital Personal Data Protection Act, 2023,” Ministry of Electronics and Information Technology, 2023.

[4] OWASP Foundation, “OWASP Top 10 for Large Language Model Applications,” 2023. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[5] Microsoft, “Presidio: Data Protection and De-identification SDK,” GitHub. [Online]. Available: https://github.com/microsoft/presidio

[6] R. Smith, “An Overview of the Tesseract OCR Engine,” in Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR), 2007, pp. 629–633.

[7] W. Wu, H. Peng and S. Yu, “YuNet: A Tiny Millisecond-level Face Detector,” Machine Intelligence Research, vol. 20, pp. 656–665, 2023.

[8] Ultralytics, “YOLO11 Documentation,” 2024. [Online]. Available: https://docs.ultralytics.com

[9] T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” in Proc. European Conf. Computer Vision (ECCV), 2014, pp. 740–755.

[10] J. Verhoeff, “Error Detecting Decimal Codes,” Mathematical Centre Tract 29, Mathematisch Centrum, Amsterdam, 1969.

[11] H. P. Luhn, “Computer for Verifying Numbers,” U.S. Patent 2 950 048, Aug. 23, 1960.

[12] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, IETF, Dec. 2011.

[13] Google, “Chrome Extensions – Manifest V3,” Chrome for Developers. [Online]. Available: https://developer.chrome.com/docs/extensions/mv3/

[14] S. Ramírez, “FastAPI Documentation.” [Online]. Available: https://fastapi.tiangolo.com

[15] Microsoft, “ONNX Runtime Web,” ONNX Runtime Documentation. [Online]. Available: https://onnxruntime.ai/docs/tutorials/web/

[16] Prince Raj, “alpha-x1: Privacy Vision Agent,” GitHub repository. [Online]. Available: https://github.com/princeraj2572/alpha-x1

Appendices

Appendix A: User Manual

Backend

```
cd privacy-vision-agent/backend
pip install -r requirements.txt
```

```
cp ../.env.example ../.env       # add your API key and choose AI_PROVIDER
uvicorn app.main:app --reload
```

Extension

```
cd privacy-vision-agent/extension
npm install
npm run build        # then load the dist folder via chrome://extensions
                     # (enable Developer mode -> Load unpacked)
```

Demo site and tests

```
cd privacy-vision-agent/demo-site && python -m http.server 8000
cd privacy-vision-agent/extension && npx vitest run
cd privacy-vision-agent/backend   && python -m pytest -v
```

Open the demo page at `http://localhost:8000`, click the extension icon to open the popup, open the side panel, enter a task and review the detections, the sanitized context and the outbound data before approving each step. Use the kill switch to stop the agent at any time.

Appendix B: Additional Results / Screenshots

Supplementary screenshots (side panel with detection overlay, split original/redacted view and outbound-data viewer) can be added here from a live run of the extension. [Insert screenshots]

Appendix C: Selected Code / Supporting Documents

Full source code, CI configuration (`.github/workflows/ci.yml`) and test suites are available in the GitHub repository: https://github.com/princeraj2572/alpha-x1.

Supervisor Evaluation / Approval

Supervisor's remarks	[For supervisor use]
Recommendation	☐ Accepted ☐ Accepted with corrections ☐ Resubmission required
Supervisor signature / date	[Signature and date]
HoD signature / date	[Signature and date, if required]